

```
In [49]: # 1. Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_curve, roc_auc_score, classification_report, con
```

```
In [34]: # 2. Load Dataset
df = pd.read_csv("german_credit_data.csv")
df.head()
```

```
Out[34]:
```

	Unnamed: 0	Age	Sex	Job	Housing	Saving accounts	Checking account	Credit amount	Duration
0	0	67	male	2	own	NaN	little	1169	6
1	1	22	female	2	own	little	moderate	5951	48
2	2	49	male	1	own	little	NaN	2096	12
3	3	45	male	2	free	little	little	7882	42
4	4	53	male	2	free	little	little	4870	24

◀  ▶

```
In [35]: # 3. Data Preprocessing

# Drop unnecessary column if exists
if "Unnamed: 0" in df.columns:
    df.drop(columns=["Unnamed: 0"], inplace=True)

# Handle missing values
df.fillna(df.median(numeric_only=True), inplace=True)

# Encode categorical variables
for col in df.select_dtypes(include='object').columns:
    df[col] = LabelEncoder().fit_transform(df[col].astype(str))

df['Default'] = (
    (df['Credit amount'] > df['Credit amount'].median()) &
    (df['Duration'] > df['Duration'].median())
).astype(int)

# Target
target_col = "Default"

X = X = df.drop(columns=['Default', 'Credit amount', 'Duration'])
y = df[target_col]

print("Target distribution:\n", y.value_counts())
```

```
Target distribution:
Default
0      650
1      350
Name: count, dtype: int64
```

```
In [36]: # 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
In [37]: # 5. Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [38]: # 6. Logistic Regression (Baseline)
log_model = LogisticRegression(max_iter=1000)

# Cross-validation (correct scoring for binary)
log_cv = cross_val_score(
    log_model, X_train_scaled, y_train, cv=5, scoring='roc_auc'
)
print("Logistic Regression CV ROC-AUC:", log_cv.mean())

# Train
log_model.fit(X_train_scaled, y_train)

# Predict probability
y_pred_prob_log = log_model.predict_proba(X_test_scaled)[:, 1]

# ROC-AUC
log_auc = roc_auc_score(y_test, y_pred_prob_log)
print("Logistic Regression ROC-AUC:", log_auc)
```

```
Logistic Regression CV ROC-AUC: 0.6428571428571429
Logistic Regression ROC-AUC: 0.7102197802197802
```

```
In [39]: # 7. Random Forest (Ensemble)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

# Cross-validation
rf_cv = cross_val_score(
    rf_model, X_train, y_train, cv=5, scoring='roc_auc'
)
print("Random Forest CV ROC-AUC:", rf_cv.mean())

# Train
rf_model.fit(X_train, y_train)

# Predict probability
y_pred_prob_rf = rf_model.predict_proba(X_test)[:, 1]

# ROC-AUC
rf_auc = roc_auc_score(y_test, y_pred_prob_rf)
print("Random Forest ROC-AUC:", rf_auc)
```

```
Random Forest CV ROC-AUC: 0.6290350274725275
Random Forest ROC-AUC: 0.7064285714285714
```

```
In [40]: # 8. Risk Classification
def classify_risk(prob):
    if prob < 0.3:
        return "Low Risk"
    elif prob < 0.7:
        return "Medium Risk"
    else:
        return "High Risk"

risk_categories = [classify_risk(p) for p in y_pred_prob_rf]
```

```
In [42]: # 9. Loan Decision Logic
def loan_decision(risk):
    if risk == "Low Risk":
        return "Approve"
    elif risk == "Medium Risk":
        return "Review"
    else:
        return "Reject"

decisions = [loan_decision(r) for r in risk_categories]
```

```
In [43]: # 10. Feature Importance
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

print("\nTop Features:\n", feature_importance.head())

# Logistic coefficients
log_coeff = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": log_model.coef_[0]
}).sort_values(by="Coefficient", ascending=False)

print("\nLogistic Coefficients:\n", log_coeff.head())
```

Top Features:

	Feature	Importance
0	Age	0.431153
6	Purpose	0.165628
4	Saving accounts	0.106600
5	Checking account	0.103188
2	Job	0.088129

Logistic Coefficients:

	Feature	Coefficient
2	Job	0.419487
4	Saving accounts	0.147173
1	Sex	0.146629
5	Checking account	-0.129701
6	Purpose	-0.163697

```
In [44]: # 11. Evaluation
y_pred_class = rf_model.predict(X_test)

print("\nClassification Report:\n", classification_report(y_test, y_pred_class))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_class))
```

Classification Report:

	precision	recall	f1-score	support
0	0.76	0.85	0.80	130
1	0.64	0.49	0.55	70
accuracy			0.72	200
macro avg	0.70	0.67	0.68	200
weighted avg	0.72	0.72	0.71	200

Confusion Matrix:

```
[[111  19]
 [ 36  34]]
```

```
In [45]: # 11. Final Output
results = pd.DataFrame({
    "Actual": y_test.values,
    "Default_Probability": y_pred_prob_rf,
    "Risk_Category": risk_categories,
    "Decision": decisions
})

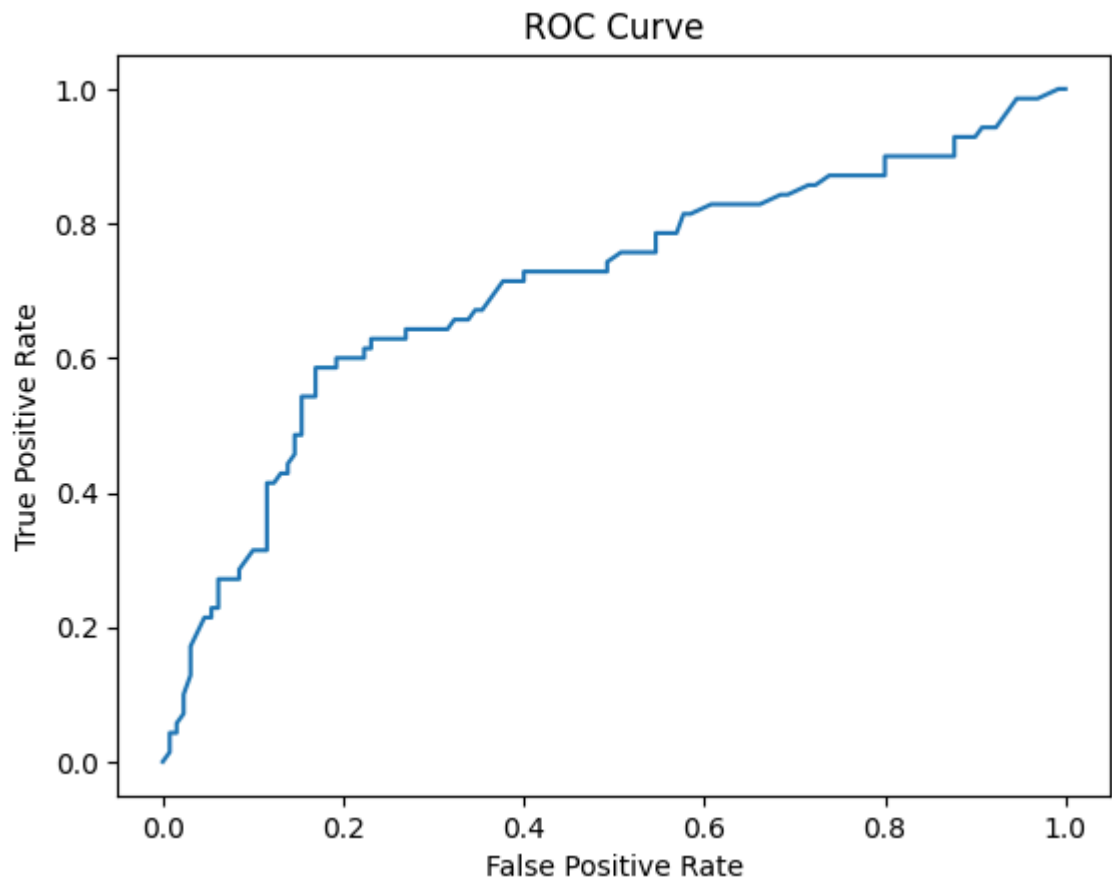
print("\nSample Results:\n", results.head())
```

Sample Results:

	Actual	Default_Probability	Risk_Category	Decision
0	0	0.338306	Medium Risk	Review
1	0	0.380000	Medium Risk	Review
2	0	0.710000	High Risk	Reject
3	0	0.790000	High Risk	Reject
4	0	0.070000	Low Risk	Approve

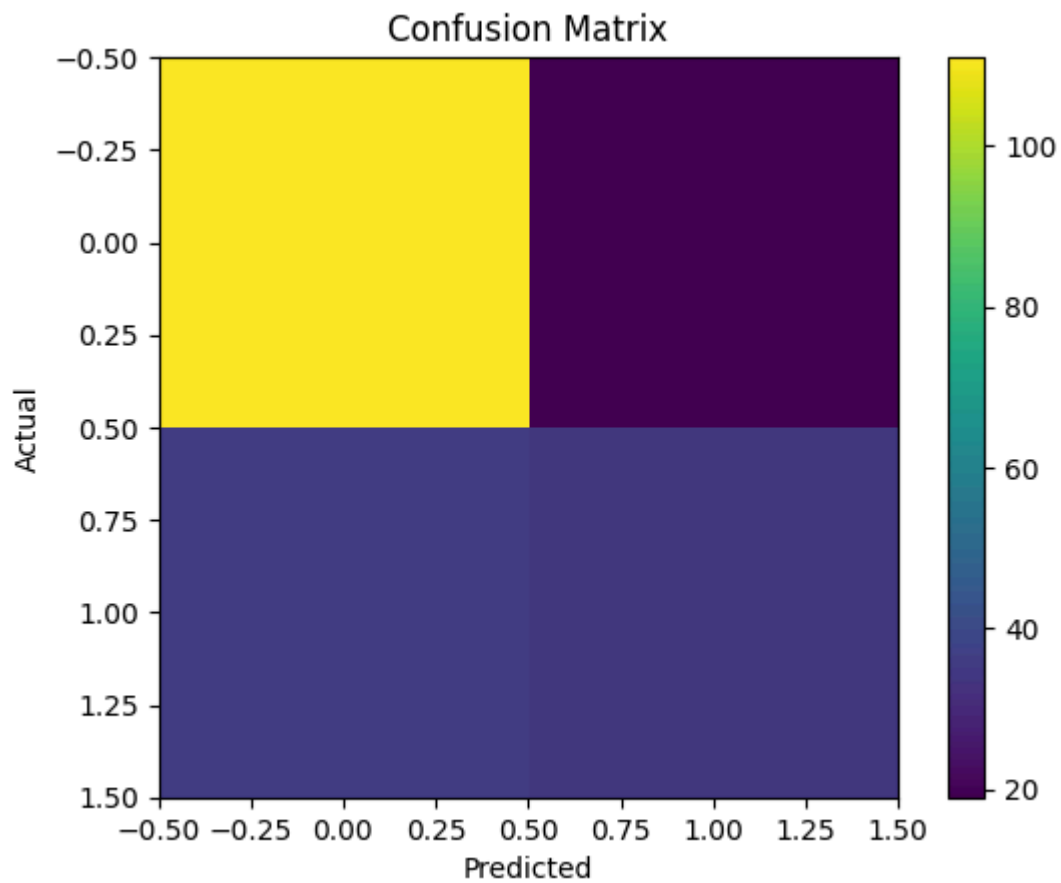
```
In [51]: #roc_curve
fpr, tpr, _ = roc_curve(y_test, y_pred_prob_rf)

plt.figure()
plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()
```



```
In [52]: #confusion matrix
cm = confusion_matrix(y_test, y_pred_class)

plt.figure()
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()
```



```
In [53]: #feature importance
feature_importance.sort_values(by="Importance").plot(
    x="Feature", y="Importance", kind="barh"
)

plt.title("Feature Importance")
plt.show()
```

